

```
import pandas as pd
import random
!pip install faker
from faker import Faker
from datetime import datetime, timedelta
```

```
fake = Faker()
```

```
random.seed(42)
```

```
# -----
# CONFIG
# -----
NUM_CLIENTS = 40
```

```
INDUSTRIES = [
    "Healthcare",
    "Finance",
    "Retail",
    "Technology",
    "Manufacturing",
    "Education",
    "Telecom"
]
```

```
COUNTRIES = [
    ("India", "Mumbai", "APAC"),
    ("Singapore", "Singapore", "APAC"),
    ("USA", "New York", "North America"),
    ("UK", "London", "Europe"),
    ("Australia", "Sydney", "APAC")
]
```

```
CLIENT_STATUS = [
    "Active",
    "At Risk",
    "Churned",
    "New"
]
```

```
LEAD_SOURCES = [
    "Call",
    "Instagram",
    "LinkedIn",
    "Social Media",
    "Website",
    "Referral",
    "Email Campaign"
```

```
]
```

```
CONTACT_MODES = [
```

```
    "Email",
```

```
    "Call",
```

```
    "Teams",
```

```
    "WhatsApp"
```

```
]
```

```
COMPANY_SIZES = [
```

```
    "Small",
```

```
    "Mid-size",
```

```
    "Enterprise"
```

```
]
```

```
# -----
```

```
# EMPLOYEE REFERENCES
```

```
# -----
```

```
# Must match employee dataset IDs
```

```
employee_ids = [f"EMP{i:03d}" for i in range(1, 101)]
```

```
employee_names = [
```

```
    fake.name() for _ in employee_ids
```

```
]
```

```
employee_lookup = dict(zip(employee_ids, employee_names))
```

```
# -----
```

```
# CLIENT MASTER
```

```
# -----
```

```
clients = []
```

```
for i in range(1, NUM_CLIENTS + 1):
```

```
    client_id = f"CLT{i:03d}"
```

```
    country, city, region = random.choice(COUNTRIES)
```

```
    company_name = fake.company()
```

```
    onboarded = fake.date_between(
```

```
        start_date="-2y",
```

```
        end_date="today"
```

```
    )
```

```
    account_manager = random.choice(employee_ids)
```

```
    csm = random.choice(employee_ids)
```

```
monthly_revenue = random.randint(50000, 1000000)
```

```
annual_revenue = monthly_revenue * 12
```

```
total_revenue = annual_revenue + random.randint(  
    100000,  
    5000000  
)
```

```
overdue = random.randint(0, 200000)
```

```
net_revenue = total_revenue - overdue
```

```
client = {  
    "client_id": client_id,  
    "client_name": company_name,  
    "primary_contact_name": fake.name(),  
    "contact_email": fake.company_email(),  
    "contact_phone": "+91" + str(  
        random.randint(6000000000, 9999999999)  
    ),  
    "industry": random.choice(INDUSTRIES),  
    "website": fake.url(),  
    "country": country,  
    "city": city,  
    "region": region,  
    "client_status": random.choice(CLIENT_STATUS),  
    "onboarded_date": onboarded,  
    "lead_source": random.choice(LEAD_SOURCES),  
  
    "account_manager": account_manager,  
    "account_manager_name": employee_lookup[account_manager],  
  
    "customer_success_manager": csm,  
    "customer_success_manager_name": employee_lookup[csm],  
  
    "preferred_contact_mode": random.choice(  
        CONTACT_MODES  
    ),  
  
    "company_size": random.choice(COMPANY_SIZES),  
  
    "employee_size": random.randint(50, 10000),  
  
    "monthly_recurring_revenue": monthly_revenue,  
    "annual_recurring_revenue": annual_revenue,  
    "total_revenue": total_revenue,
```

```

    "average_monthly_spend": round(
        total_revenue / 12, 2
    ),

    "revenue_band": random.choice(
        ["Low", "Medium", "High"]
    ),

    "overdue_amount": overdue,

    "net_revenue": net_revenue,

    "customer_lifetime_value": random.randint(
        500000,
        50000000
    )
}

```

```

clients.append(client)

```

```

client_df = pd.DataFrame(clients)

```

```

# -----
# CLIENT HEALTH
# -----
health_rows = []

```

```

for idx, client in enumerate(clients, start=1):

```

```

    health_score = random.randint(50, 100)

```

```

    churn_risk = random.randint(1, 100)

```

```

    health_rows.append({
        "client_id": client["client_id"],
        "health_score": health_score,
        "churn_risk_score": churn_risk,
        "engagement_score": random.randint(40, 100),
        "satisfaction_score": random.randint(40, 100),
        "support_score": random.randint(40, 100),
        "renewal_probability": random.randint(40, 100),
    })

```

```

    "risk_level": (
        "High"
        if churn_risk > 75
        else "Medium"
        if churn_risk > 40
        else "Low"
    )

```

```

    ),
    "expansion_opportunity_flag": random.choice(
        ["Yes", "No"]
    )
})

```

```
health_df = pd.DataFrame(health_rows)
```

```
# -----
```

```
# CLIENT HISTORY
```

```
# -----
```

```
history_rows = []
```

```
for idx, client in enumerate(clients, start=1):
```

```

    previous_revenue = random.randint(
        100000,
        1000000
    )

```

```
current_revenue = client["total_revenue"]
```

```

history_rows.append({
    "historical_id": idx,
    "client_id": client["client_id"],
    "previous_tier": random.choice(
        ["Silver", "Gold"]
    ),
    "current_tier": random.choice(
        ["Gold", "Platinum"]
    ),
    "previous_revenue": previous_revenue,
    "current_revenue": current_revenue,

```

```

    "change_reason": random.choice([
        "Upsell",
        "Renewal",
        "Expansion",
        "Contract Revision"
    ]),

```

```

    "change_type": random.choice([
        "Upgrade",
        "Downgrade",
        "Manager Change"
    ]),

```

```

        "valid_from": fake.date_between(
            start_date="-2y",
            end_date="-1y"
        ),

        "valid_to": fake.date_between(
            start_date="-1y",
            end_date="today"
        ),

        "retention_status": random.choice(
            ["Renewed", "Lost"]
        ),

        "revenue_change": (
            current_revenue - previous_revenue
        )
    })

```

```

history_df = pd.DataFrame(history_rows)

```

```

# -----
# CLIENT CONTACTS
# -----
contacts_rows = []

```

```

for i in range(1, 121):

```

```

    client = random.choice(clients)

```

```

    contacts_rows.append({
        "contact_id": f"CNT{i:03d}",
        "client_id": client["client_id"],
        "contact_name": fake.name(),
        "designation": random.choice([
            "CTO",
            "CEO",
            "Manager",
            "Director",
            "VP"
        ]),
        "department": random.choice([
            "Technology",
            "Finance",
            "Operations",
            "Sales"
        ]),
        "email": fake.company_email(),
    })

```

```

        "phone": "+91" + str(
            random.randint(6000000000, 9999999999)
        ),
        "decision_maker_flag": random.choice(
            ["Yes", "No"]
        ),
        "seniority_level": random.choice([
            "Executive",
            "Director",
            "Manager"
        ]),
        "preferred_contact_time": random.choice([
            "Morning",
            "Afternoon",
            "Evening"
        ]),
        "communication_preference": random.choice(
            CONTACT_MODES
        )
    })
}

```

```

contacts_df = pd.DataFrame(contacts_rows)

```

```

# -----
# CLIENT INTERACTIONS
# -----
interaction_rows = []

```

```

for i in range(1, 301):

```

```

    client = random.choice(clients)

```

```

    employee_id = random.choice(employee_ids)

```

```

    interaction_date = fake.date_between(
        start_date="-1y",
        end_date="today"
    )

```

```

    followup_required = random.choice(
        ["Yes", "No"]
    )

```

```

    interaction_rows.append({
        "interaction_id": f"INT{i:04d}",
        "client_id": client["client_id"],
        "contact_name": fake.name(),
    })

```

```

"employee_id": employee_id,
"employee_name": employee_lookup[employee_id],

"interaction_type": random.choice([
    "Call",
    "Meeting",
    "Demo",
    "Email",
    "Social Media"
]),

"interaction_date": interaction_date,

"interaction_duration_minutes": random.randint(
    10,
    120
),

"meeting_channel": random.choice([
    "Zoom",
    "Teams",
    "In-person",
    "Phone"
]),

"interaction_outcome": random.choice([
    "Positive",
    "Neutral",
    "Negative"
]),

"followup_required": followup_required,

"next_followup_date": (
    interaction_date + timedelta(days=7)
    if followup_required == "Yes"
    else None
),

"response_time_hours": round(
    random.uniform(1, 72),
    1
)
})

interaction_df = pd.DataFrame(interaction_rows)

# -----

```



```

# CONTRACTS
# -----
contract_rows = []

for i in range(1, 61):

    client = random.choice(clients)

    start_date = fake.date_between(
        start_date="-2y",
        end_date="-1y"
    )

    end_date = start_date + timedelta(
        days=random.randint(180, 720)
    )

    contract_rows.append({
        "contract_id": f"CON{i:03d}",
        "client_id": client["client_id"],

        "contract_type": random.choice([
            "MSA",
            "Fixed Bid",
            "Time & Material",
            "Support"
        ]),

        "project_name": fake.bs().title(),

        "project_start_date": start_date,
        "project_end_date": end_date,

        "contract_start_date": start_date,
        "contract_end_date": end_date,

        "total_contract_value": random.randint(
            100000,
            10000000
        ),

        "renewal_date": end_date - timedelta(days=30),

        "auto_renew_flag": random.choice([
            "Yes",
            "No"
        ]),
    })

```

```

        "payment_status": random.choice([
            "Paid",
            "Pending",
            "Overdue"
        ]),

        "billing_frequency": random.choice([
            "Monthly",
            "Quarterly",
            "Annual"
        ]),

        "contract_status": random.choice([
            "Active",
            "Expired",
            "Under Review"
        ])
    })
})

```

```
contract_df = pd.DataFrame(contract_rows)
```

```

# -----
# SATISFACTION
# -----
survey_rows = []

```

```
for i in range(1, 81):
```

```
    client = random.choice(clients)
```

```
    nps = random.randint(1, 10)
```

```

    category = (
        "Promoter"
        if nps >= 9
        else "Passive"
        if nps >= 7
        else "Detractor"
    )

```

```

survey_rows.append({
    "survey_id": f"SRV{i:03d}",
    "client_id": client["client_id"],

    "net_promoter_score": nps,

    "customer_category": category,

```

```

"feedback_comment": fake.sentence(),

"service_quality_score": random.randint(
    1,
    10
),

"renewal_intent": random.choice([
    "High",
    "Medium",
    "Low"
]),

"recommendation_likelihood": random.randint(
    1,
    10
),

"survey_date": fake.date_between(
    start_date="-1y",
    end_date="today"
)
})

```

```
survey_df = pd.DataFrame(survey_rows)
```

```

# -----
# CLIENT REVENUE TABLE
# -----
revenue_rows = []

```

```
for i in range(1, 301):
```

```
    client = random.choice(clients)
```

```

    revenue_amount = random.randint(
        50000,
        2000000
    )

```

```
    tax = round(revenue_amount * 0.18, 2)
```

```
    net = revenue_amount - tax
```

```

    revenue_rows.append({
        "revenue_id": f"REV{i:04d}",
        "client_id": client["client_id"],
        "client_name": client["client_name"],
    })

```

```
"project_id": f"PRJ{random.randint(1,20):03d}",

"project_name": fake.bs().title(),

"invoice_id": f"INV{i:04d}",

"contract_id": f"CON{random.randint(1,60):03d}",

"revenue_period": fake.month_name() + "-2025",

"revenue_date": fake.date_between(
    start_date="-1y",
    end_date="today"
),

"revenue_amount": revenue_amount,

"currency": random.choice([
    "INR",
    "USD",
    "SGD"
]),

"revenue_type": random.choice([
    "Recognized",
    "Deferred",
    "Accrued"
]),

"revenue_stream": random.choice([
    "Recurring",
    "One-time"
]),

"monthly_recurring_revenue_mrr": round(
    revenue_amount / 12,
    2
),

"annual_recurring_revenue_arr": revenue_amount,

"gross_revenue": revenue_amount,

"net_revenue": net,

"tax_amount": tax,
```

```
"profit_margin": round(
    random.uniform(10, 45),
    2
),

"recognition_date": fake.date_between(
    start_date="-1y",
    end_date="today"
),

"payment_status": random.choice([
    "Paid",
    "Partial",
    "Pending",
    "Overdue"
]),

"outstanding_amount": random.randint(
    0,
    500000
),

"collection_days": random.randint(
    5,
    90
),

"revenue_growth_percentage": round(
    random.uniform(-10, 40),
    2
),

"revenue_band": random.choice([
    "Low",
    "Medium",
    "High"
]),

"forecast_revenue": random.randint(
    100000,
    3000000
),

"upsell_revenue": random.randint(
    0,
    500000
),
```

```

        "country": client["country"],

        "billing_frequency": random.choice([
            "Monthly",
            "Quarterly",
            "Annual"
        ])
    })
})

revenue_df = pd.DataFrame(revenue_rows)

# -----
# EXPORT CSV
# -----
client_df.to_csv("client_master.csv", index=False)

health_df.to_csv(
    "client_health.csv",
    index=False
)

history_df.to_csv(
    "client_history.csv",
    index=False
)

contacts_df.to_csv(
    "client_contacts.csv",
    index=False
)

interaction_df.to_csv(
    "client_interactions.csv",
    index=False
)

contract_df.to_csv(
    "client_contracts.csv",
    index=False
)

survey_df.to_csv(
    "client_satisfaction.csv",
    index=False
)

revenue_df.to_csv(
    "client_revenue.csv",

```

```
        index=False
    )

    print("Client CSV files generated successfully!")
```